Reinforcement learning

10-701 Introduction to Machine Learning
Geoff Gordon

Estimating the total reward of a policy

- Naive method: run some trajectories with agent acting according to policy π, average their total rewards
- Insight: the actual sequence of states visited is informative — use it to reduce variance
- We'll estimate rewards for each state, use relationships among states to improve accuracy
- ***Defn**:* the ***value function*** of π maps state $\rightarrow$ total expected future reward if we are at that state following π

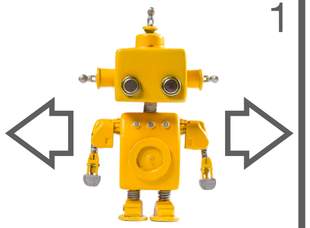
$$V^\pi(s) = \mathbb{E}\left[\sum_{i=t,\,t+1,\,\ldots} r_i \,\middle|\, s_t = s, \text{following } \pi\right]$$

  - for now, assume expectation exists; we'll return to this

Value function example

For simplicity, suppose we know the exact environment

Environment

1	2	3	4	5
				
r=–1	r=–1	r=–10	r=–1	r=0

Policy π: always move right

Trajectory ends with any action from 5

Table of V^π

1	2	3	4	5
–13	–12	–11	–1	0

Computing the value function

$$
\begin{aligned}
V^\pi(s) &= \mathbb{E}(r_1 + r_2 + r_3 + \ldots \mid s_1 = s, \text{following } \pi) \\
&= \mathbb{E}(r_1 + \mathbb{E}(r_2 + r_3 + \ldots \mid s_2 = \text{observed } s_2, \pi) \mid s_1 = s, \pi) \\
&= \mathbb{E}(r_1 + V^\pi(s_2) \mid s_1 = s, \pi) \\
&= \sum_a \pi(a \mid s) \left[r(s,a) + \sum_{s'} T(s' \mid s, a) V^\pi(s') \right]
\end{aligned}
$$

note: T is substochastic — handles end of trajectory automatically

- Split $V^\pi(s)$ into first step and subsequent steps
- Use law of iterated expectations + Markov property
- Result: V^π is expressed recursively (in terms of itself)
  - called the ***Bellman equation***

Solving the Bellman equation

$$V^\pi(s) = \sum_a \pi(a \mid s) \left[r(s,a) + \sum_{s'} T(s' \mid s, a) V^\pi(s') \right]$$

- This is a system of linear equations in $|\mathcal{S}|$ variables (the numbers $V^\pi(s)$ for all $s \in \mathcal{S}$)
- For tabular case, can construct and solve this system with `np.linalg.lstsq` or similar tool
- Instead we'll use ***dynamic programming***
  - general strategy for anything defined recursively
  - in our case: treat equation $V = f(V)$ as assignment $V \leftarrow f(V)$ and repeat
- For Bellman eq, dynamic programming converges quickly to a good-enough answer, and provides a basis for algorithms that generalize beyond the tabular case

policy always →

Value iteration for V^π

Dynamic programming:
replace = by ← in Bellman equation

Environment

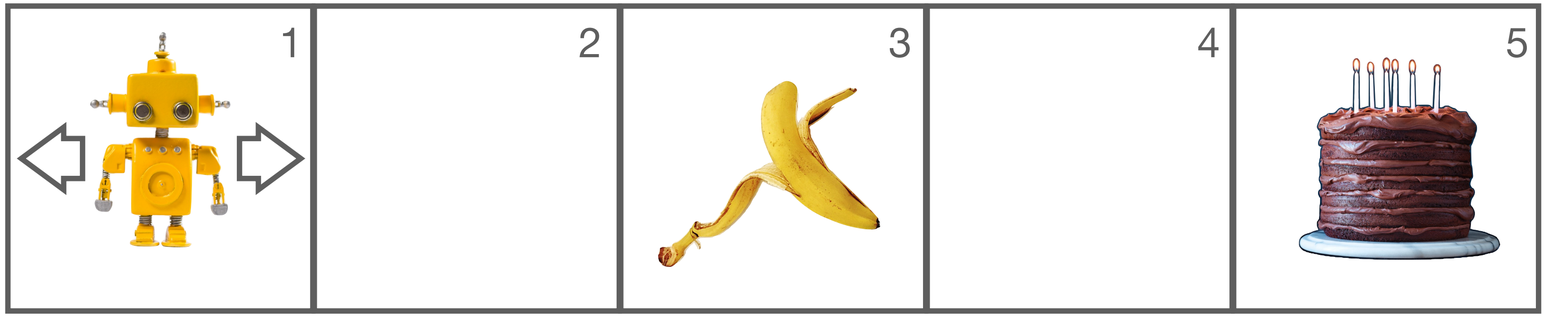
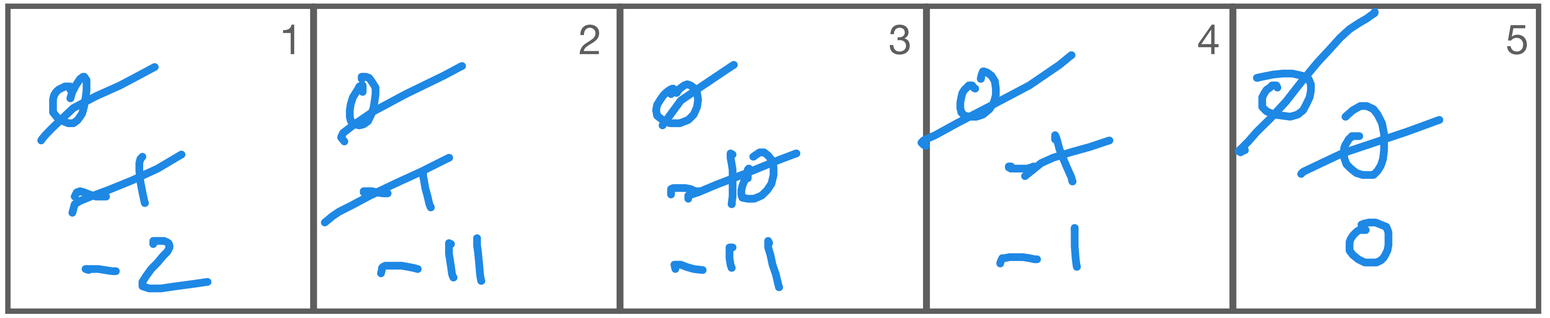


Table of V^π



- Given: $r(s,a)$, $T(s' \mid s,a)$ (exact or learned)
- Initialize $V^\pi(s)$ arbitrarily (e.g., to 0 for all s)

or warm start if we know one

- Repeat until converged
  - for each state s (in parallel or in an arbitrary order)

$$V^\pi(s) \leftarrow \sum_a \pi(a \mid s)\left[r(s,a) + \sum_{s'} T(s' \mid s,a) V^\pi(s')\right]$$

Temporal difference (TD) learning

same idea, but from data instead of from environment model

Data $1 \xrightarrow{-1} 2 \xrightarrow{-1} 3 \xrightarrow{-10} 4 \xrightarrow{-1} 5 \xrightarrow{0}$ goal (π: always R)

Table of V^π

1	2	3	4	5
0	-3	-5	-1/2	0

- Given: observed transitions from π, (s_t, r_t, s_{t+1})
- Initialize $V^\pi(s)$ arbitrarily (e.g., to 0 for all s)
- For each minibatch of transitions:
  - disable gradients and compute targets for the minibatch: $v \leftarrow r + \gamma V(s')$ for all transitions (s, r, s')
  - after all targets are computed, re-enable gradients and train each $V^\pi(s)$ towards its target v

like computing RHS of assignment

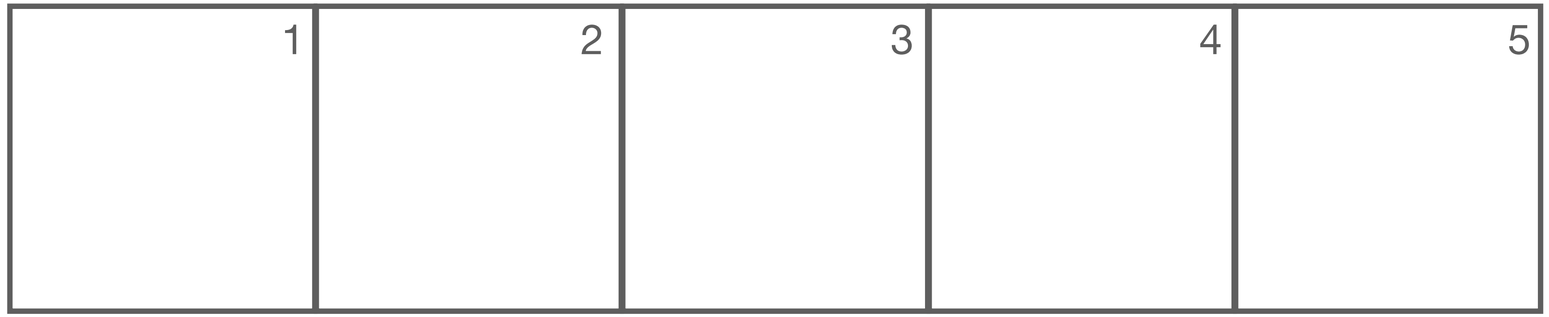
like doing the assignment

Temporal difference (TD) learning

here `V` is the tabular value function, and `s`, `r` and `next_s` are the minibatch

Data $1 \to^{-1} 2 \to^{-1} 3 \to^{-10} 4 \to^{-1} 5 \to^{0}$ goal (π: always R)

Table of V^π




```
with torch.no_grad():
  target = r + gamma * V[next_s]
loss = 0.5 * ((target - V[s]) ** 2).sum()
opt.zero_grad()
loss.backward()
opt.step()
```

Temporal difference (TD) learning

Data $1 \rightarrow^{-1} 2 \rightarrow^{-1} 3 \rightarrow^{-10} 4 \rightarrow^{-1} 5 \rightarrow^{0}$ goal (π: always R)

Table of V^π

1	2	3	4	5

- Base case is important: if we see (s, r, goal), train $V^\pi(s)$ toward r (keep values anchored)
- Minibatch size interpolates between sequential and parallel VI behavior
- If we forget to stop gradients, it's a different algorithm (and doesn't seem to extend as well beyond tabular)

Optimal value function

Def'n:

$$V^*(s) = \max_\pi \mathbb{E}(r_1 + r_2 + r_3 + \ldots \mid s_1 = s, \text{following } \pi)$$

$$V^*(s) = \max_a \mathbb{E}\left[r_1 + \max_\pi \mathbb{E}(r_2 + r_3 + \ldots \mid s_2 = \text{observed } s_2, \pi) \mid s_1 = s, a_1 = a)\right]$$

$$= \max_a \mathbb{E}\left[r_1 + V^*(s_2) \mid s_1 = s, a_1 = a\right]$$

$$= \max_a \left[r(s,a) + \sum_{s'} T(s' \mid s, a) V^*(s')\right]$$

- Decompose $V^*(s)$ into first step and subsequent steps, using law of iterated expectations as above
- Split $\max_\pi$ into separate max over a_1 and over π for $t = 2$ onward
- Result: V^* expressed recursively (another Bellman eq)

Value iteration for V^*

Replace = by ← in Bellman equation

Environment

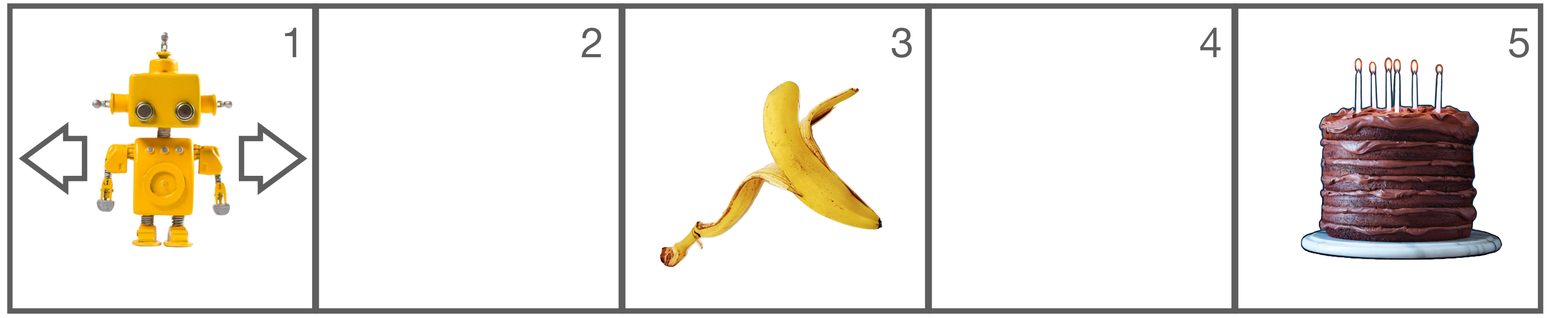
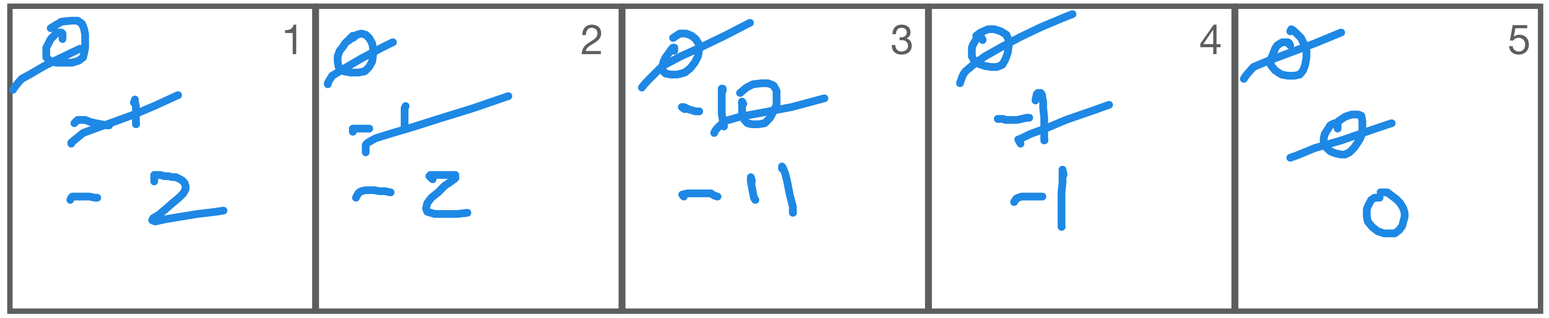


Table of V^π



- Given: $r(s,a)$, $T(s' \mid s,a)$
- Initialize $V^*(s)$ arbitrarily (e.g., to 0 for all s)
- Repeat until converged
  - for each state s (in parallel or in an arbitrary order)

$$V^*(s) \leftarrow \max_a \left[r(s,a) + \sum_{s'} T(s' \mid s,a) V^*(s') \right]$$

Value iteration for V^*

Replace = by ← in Bellman equation

There's no temporal difference version of value iteration for V (why not?)*

Environment

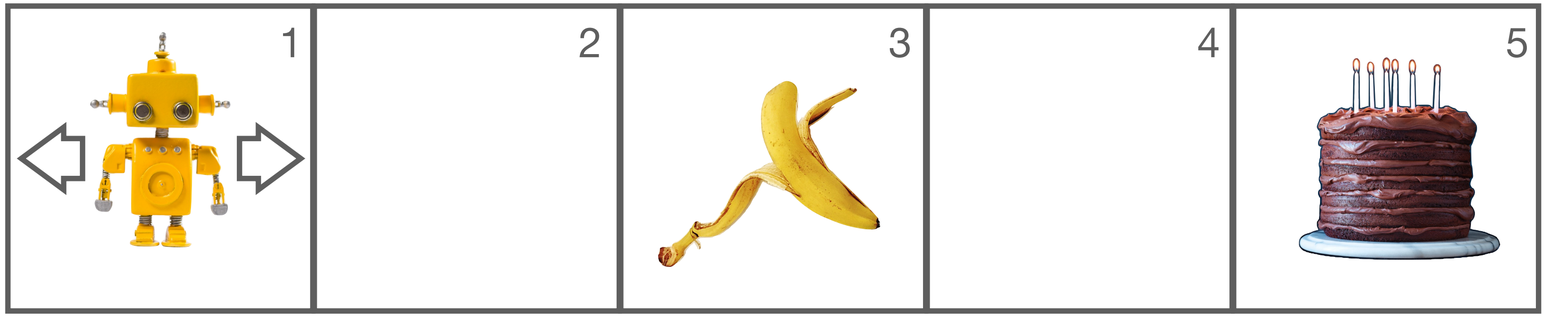


Table of V^π

1	2	3	4	5

- Given: $r(s,a)$, $T(s' \mid s,a)$
- Initialize $V^*(s)$ arbitrarily (e.g., to 0 for all s)
- Repeat until converged
  - for each state s (in parallel or in an arbitrary order)

$$V^*(s) \leftarrow \max_a \left[r(s,a) + \sum_{s'} T(s' \mid s,a) V^*(s') \right]$$

Greedy policy

- V^π or V^* plus environment model → design an agent
- Look at RHS of Bellman: $r(s,a) + \sum_{s'} T(s' \mid s,a) V(s')$
  - if agent is at s, this is predicted total future reward for choosing action a
- ***Greedy*** policy: choose a to maximize this predicted reward (separately at each state)
  - a one-step lookahead search to choose best action
  - also makes sense to look farther ahead: k-step greedy
- ***Theorem:*** the greedy policy for V^π is always at least as good as π — and strictly better unless π is already optimal (i.e., unless $V^\pi = V^*$)

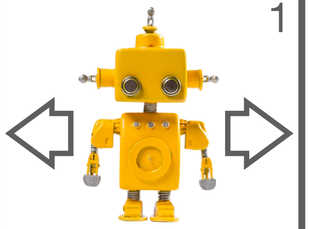
The need for greed

- Need for greedy policy motivates two kinds of algorithm
- We could track a policy explicitly, improve it over time
  - e.g., ***policy iteration****:* alternately compute V^π and set π to greedy policy
- Or we could try to make it easier to read off the greedy policy from our computed values
  - avoid the need for an explicit environment model (next slides)

Q function

Q^π or Q^* makes it trivial to compute the greedy policy — don't need to know $r(s,a)$ or $T(s' \mid s,a)$

Policy π: always move right

1	2	3	4	5
				
r=–1	r=–1	r=–10	r=–1	r=0

Q^π

	1	2	3	4	5
L	–14	–14	-22	–12	0
R	–13	–12	–11	–1	0

- ***Def'n***: $Q(s,a) = r(s,a) + \sum_{s'} T(s' \mid s,a) V(s')$
  - $Q^\pi(s,a)$ or $Q^*(s,a)$ = how much total reward if we start at $s_1 = s$, choose $a_1 = a$, continue with π or π^*
  - precalculated one-step lookahead on V
- Called the Q function or action-value function

Dynamic programming for Q-function

- Just like the value function, ∃ dynamic programming algorithms for Q^π or Q^*
- From (given or learned) environment model:
  - for either Q^π or Q^*: ***Q-iteration***
- From trajectories:
  - for Q^*: ***Q-learning***
  - for Q^π: ***SARSA***

Q-iteration

Environment

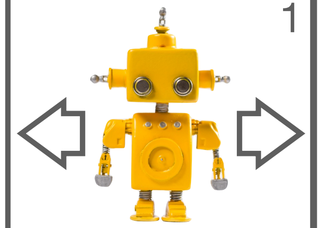
1	2	3	4	5
				
r=–1	r=–1	r=–10	r=–1	r=0

Table of Q^*

	1	2	3	4	5
L					
R					

note base case:
$Q(\text{next}, \cdot) = 0$ if done

- Given: $r(s, a)$, $T(s' \mid s, a)$
- Initialize $Q^*(s, a)$ or $Q^\pi(s, a)$ arbitrarily (e.g., to 0)
- Repeat until converged
  - for each state s and action a (in parallel or arbitrary order)

$$Q^*(s, a) \leftarrow r(s, a) + \sum_{s'} T(s' \mid s, a) \max_{a'} Q^*(s', a')$$

$$Q^\pi(s, a) \leftarrow r(s, a) + \sum_{s', a'} T(s' \mid s, a) \pi(a' \mid s') Q^\pi(s', a')$$

Q-learning

same idea, but from data instead of from environment model

base case: important to train $Q^*(5, L/R)$ toward 0

Data $1 \rightarrow^{-1} 2 \rightarrow^{-1} 3 \rightarrow^{-10} 4 \rightarrow^{-1} 5 \rightarrow^{0}$ goal (π: always R)

Table of Q^*

	1	2	3	4	5
L	0	0	0	0	0
R	0	0 -1/2	0	0	0

- Given: observed transitions from π, (s_t, a_t, r_t, s_{t+1})
- Initialize $Q^*(s, a)$ arbitrarily (e.g., to 0)
- For each minibatch of transitions
  - disable gradients; compute targets $r + \max_{a'} Q^*(s', a')$ for transitions (s, a, r, s')
  - after all targets are computed, re-enable gradients and update each $Q^*(s, a)$ by SGD towards its target

base case: important to train $Q^\pi(5, L/R)$ toward 0

Data $1 \rightarrow^{-1} 2 \rightarrow^{-1} 3 \rightarrow^{-10} 4 \rightarrow^{-1} 5 \rightarrow^{0}$ goal (π: always R)

Table of Q^π

	1	2	3	4	5
L					
R		-1/2			

SARSA

uses actual next action instead of sum or max over next action

- Given: observed transitions from π, $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$
- Initialize $Q^\pi(s, a)$ arbitrarily (e.g., to 0)
- For each minibatch of transitions
  - stop gradients; compute targets $r + Q^\pi(s', a')$ for transitions s, a, r, s', a'
  - after all targets are computed, update each $Q^\pi(s, a)$ by SGD towards its target

base case: important to train $Q^\pi(5, L/R)$ toward 0

Data $1 \rightarrow^{-1} 2 \rightarrow^{-1} 3 \rightarrow^{-10} 4 \rightarrow^{-1} 5 \rightarrow^{0}$ goal (π: always R)

Table of Q^π

	1	2	3	4	5
L					
R					

SARSA

uses actual next action instead of sum or max over next action

- Given: observed transitions from π, $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$
- Initialize $Q^\pi(s, a)$ arbitrarily (e.g., to 0)
- For each minibatch of transitions
  - stop gradients; compute targets $r + Q^\pi(s', a')$ for transitions s, a, r, s', a'
  - after all targets are computed, update each $Q^\pi(s, a)$ by SGD towards its target

Discounting

- Would you rather have $10 today or $11 a year from today?

Discounting

- Would you rather have $10 today or $11 a year from today?
- The future is uncertain: “better an egg today than a hen tomorrow”
- Math: suppose our model is right w.p. $\gamma \in (0,1)$ independently on each step
  - if right, business as usual
  - if wrong, all future costs or rewards are ***independent*** of my actions and observations so far

    $$V^\pi(s) = \mathbb{E}(r_1 + \gamma V^\pi(s_2) + (1-\gamma)\,\text{const} \mid s_1 = s, \pi)$$

    and similarly for V^*, Q^π, Q^*

    we can drop the constant
  - unrolling, expectation of $r_1 + \gamma r_2 + \gamma^2 r_3 + \ldots$
    - “expected discounted total return” and a new Bellman eq

Discounted Bellman equations

$$V^{\pi}(s) = \sum_a \pi(a \mid s)\left[r(s,a) + \gamma \sum_{s'} T(s' \mid s,a) V^{\pi}(s')\right]$$

$$V^{*}(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} T(s' \mid s,a) V^{*}(s')\right]$$

$$Q^{\pi}(s,a) = r(s,a) + \gamma \sum_{s',a'} T(s' \mid s,a)\pi(a' \mid s') Q^{\pi}(s',a')$$

$$Q^{*}(s,a) = r(s,a) + \gamma \sum_{s'} T(s' \mid s,a) \max_{a'} Q^{*}(s',a')$$

- Or sampled: e.g., target for SARSA is $r + \gamma Q^{\pi}(s',a')$

Discounts everywhere

- Big practical advantages of discounting:
  - sum over time always converges (resolves worry from earlier about existence of expectation)
  - dynamic programming algorithms become more stable
- So it's common to use a discount factor even if it's a less-accurate model of the world
  - or to use a too-strong (too-small) discount factor
- Luckily, optimal policy is often not too sensitive to γ
  - but if we see weird behavior, remember to check whether a too-small γ is the cause
  - failure mode: excessive impatience (chooses a smaller reward soon, instead of waiting for a larger reward)

Exploration

note: adding a bit of randomness to policy (ϵ-greedy) is *not* enough to explore

- Dynamic programming does temporal credit assignment
- For explore-exploit: need to intentionally collect data in data-poor areas of environment

[Brafman & Tennenholtz]

- Example strategy: exploration bonus (***R-max*** algorithm)
  - if enough data that we're ϵ-confident in rewards and transitions for s, a, learn model as usual
  - else, pretend s, a is utopia: big reward, trajectory ends
  - run value iteration (or …) and follow greedy policy
  - if any utopia s, a is reachable w/ reasonable cost and time, we'll go there (and therefore collect more data)
  - else s, a didn't matter for optimal policy

The end of the world

A finite-length trajectory, from start to end of world, is an ***episode***

- Sometimes task ends: win or lose the game, reach the goal
  - learn over a sequence of trials
- Several possible setups for the end of the world:
  - Always same number of steps: *fixed-horizon*

$$\min_{\text{policy } \pi \in \Pi} \mathbb{E}_{\text{trajectory } \tau|\pi} [c_1 + c_2 + \ldots + c_T]$$

  - Task ends when we reach *terminal state*, all intermediate costs positive (rewards negative): *stochastic shortest paths (SSP)*

$$\min_{\text{policy } \pi \in \Pi} \mathbb{E}_{\text{trajectory } \tau|\pi} [c_1 + c_2 + \ldots + c_{T(\tau)}]$$

  - Task continues wp $\gamma \in [0,1)$ after each time step: *discounted*

$$\min_{\text{policy } \pi \in \Pi} \mathbb{E}_{\text{trajectory } \tau|\pi} [c_1 + \gamma c_2 + \gamma^2 c_3 + \ldots]$$

- World never ends: *infinite-horizon*
  - Must do something to prevent infinite costs/rewards: *discounted* or *average*

The end of the world

A finite-length trajectory, from start to end of world, is an ***episode***

- Sometimes task ends: win or lose the game, reach the goal
  - learn over a sequence of trials
- Several possible setups for the end of the wor
  - Always same number of steps: *fixed-h*

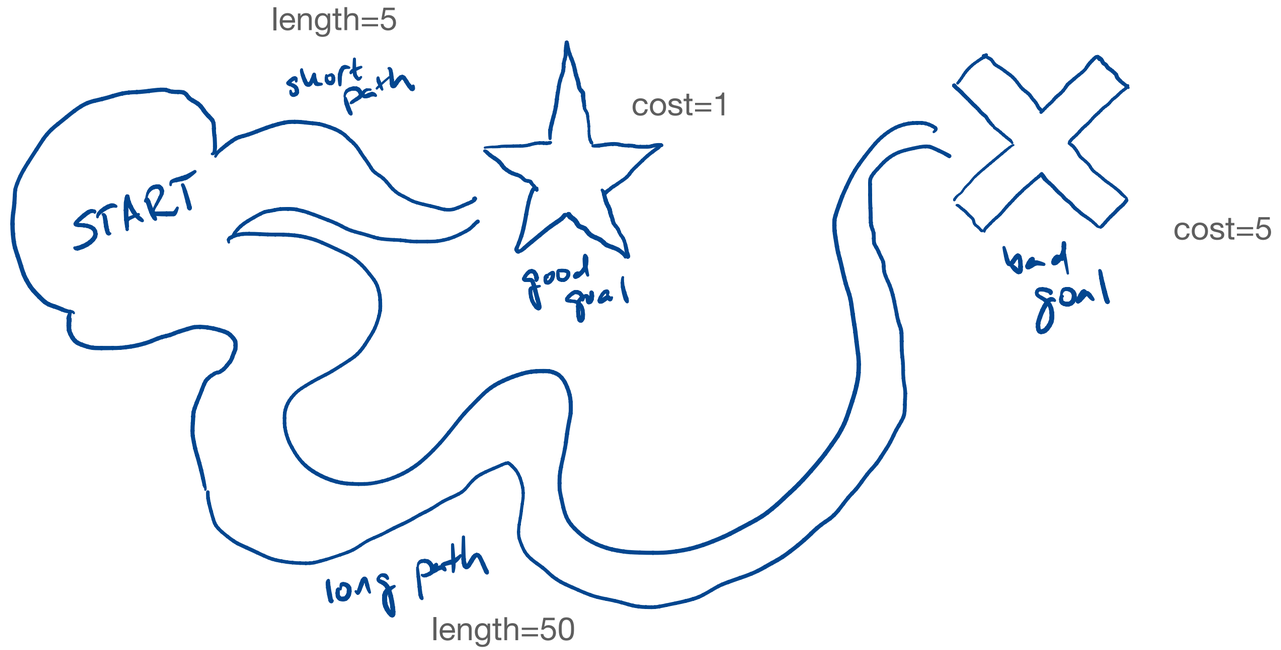
    $\min_{\text{policy } \pi \in \Pi} \mathbb{E}_{\text{trajectory } \tau|\pi} [\quad + c_T]$
  - Task ends when we re*ate*, all intermediate costs positive (rewards *nastic shortest paths (SSP)*

    $\min$ $\text{ctory } \tau|\pi$ $[c_1 + c_2 + \ldots + c_{T(\tau)}]$
  - Task con p $\gamma \in [0,1)$ after each time step: *discounted*

    $\min_{\text{policy } \pi \in \Pi} \mathbb{E}_{\text{trajectory } \tau|\pi} [c_1 + \gamma c_2 + \gamma^2 c_3 + \ldots]$
- World never ends: *infinite-horizon*
  - Must do something to prevent infinite costs/rewards: *discounted* or *average*

All setups are fine — but important to pick one! (common source of bugs)

Example bug



- Cost: 0.01 per step, plus cost of goal reached
- Desired policy: take the short path to the good goal
- Correct: treat as SSP, costs are 1.05 and 5.5
- Incorrect: use discount factor
  - good: goal cost $1 \cdot 0.95^5 = 0.77$, discounted cost of path $= 0.045$
  - bad: goal cost $5 \cdot 0.95^{50} = 0.38$, discounted cost of path $= 0.18$
  - bad goal seems better!

Termination vs. truncation

- Many software packages (e.g., OpenAI Gym) return flags from the next-state function — i.e., function that maps $(s, a) \mapsto$ sample of (r, s')
  - terminated: we reached end-of-world, episode is over, s' is not a valid input for further calls to next-state function
  - truncated: we cut off the trajectory early, e.g., because we asked for a batch of at most 100 transitions — s' would have been valid but we didn't ask to keep going

Termination vs. truncation

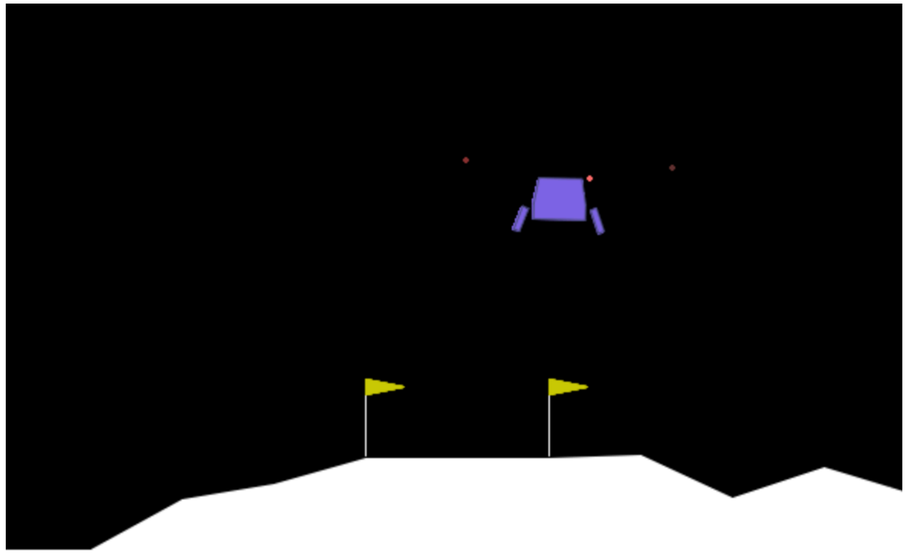
- Correct handling:
  - terminated: treat value of s' as 0, e.g., train $Q^\pi(s, a)$ toward $r + \gamma 0$
  - truncated: construct training targets as we would normally, e.g., train $Q^\pi(s, a)$ toward $r + \gamma Q^\pi(s', a')$
  - check the exact semantics of your package: plenty of off-by-one bugs are possible, e.g., is s or s' the last valid state, does s' already start the next trajectory
  - Gym is not even self-consistent: semantics vary by version and by API within Gym

Scaling up RL

10-701 Introduction to Machine Learning

Geoff Gordon

Scaling up RL



Lunar lander

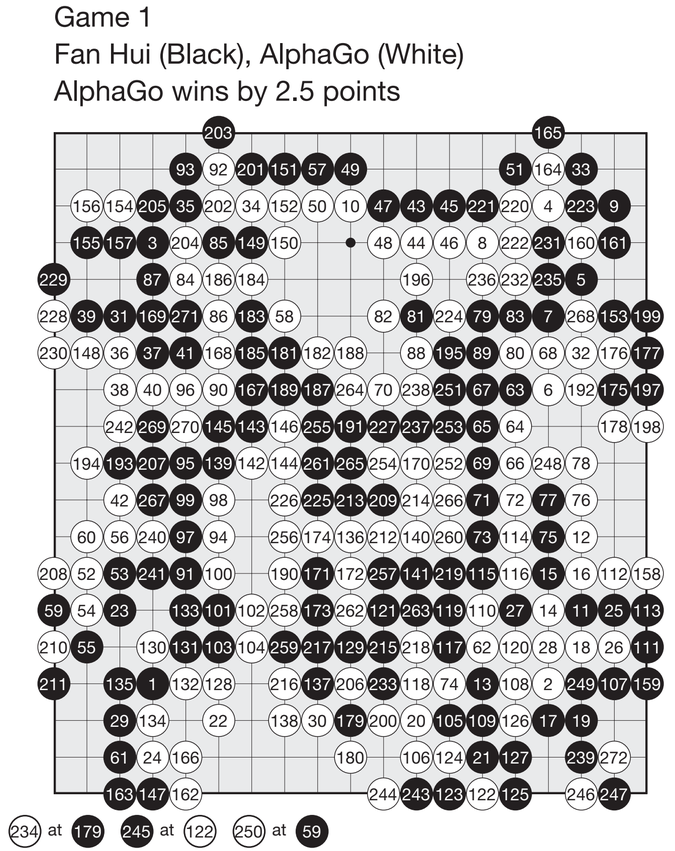
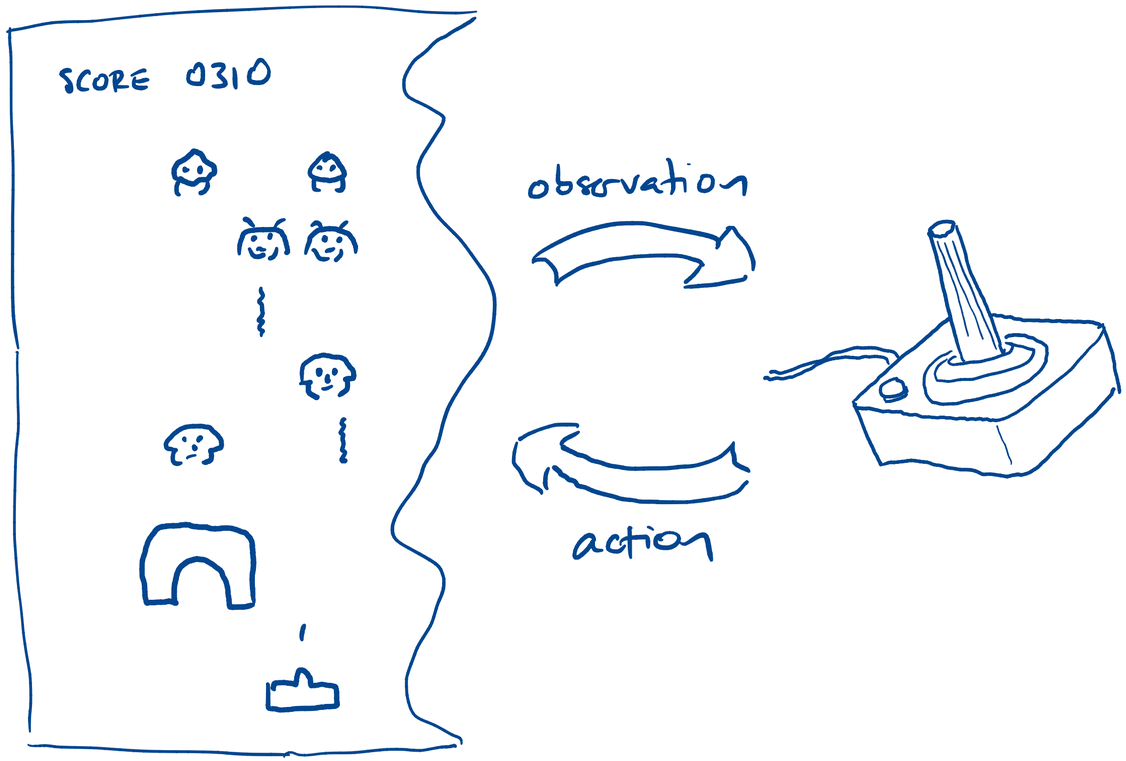


Image credit: Silver et al., Nature, 2016

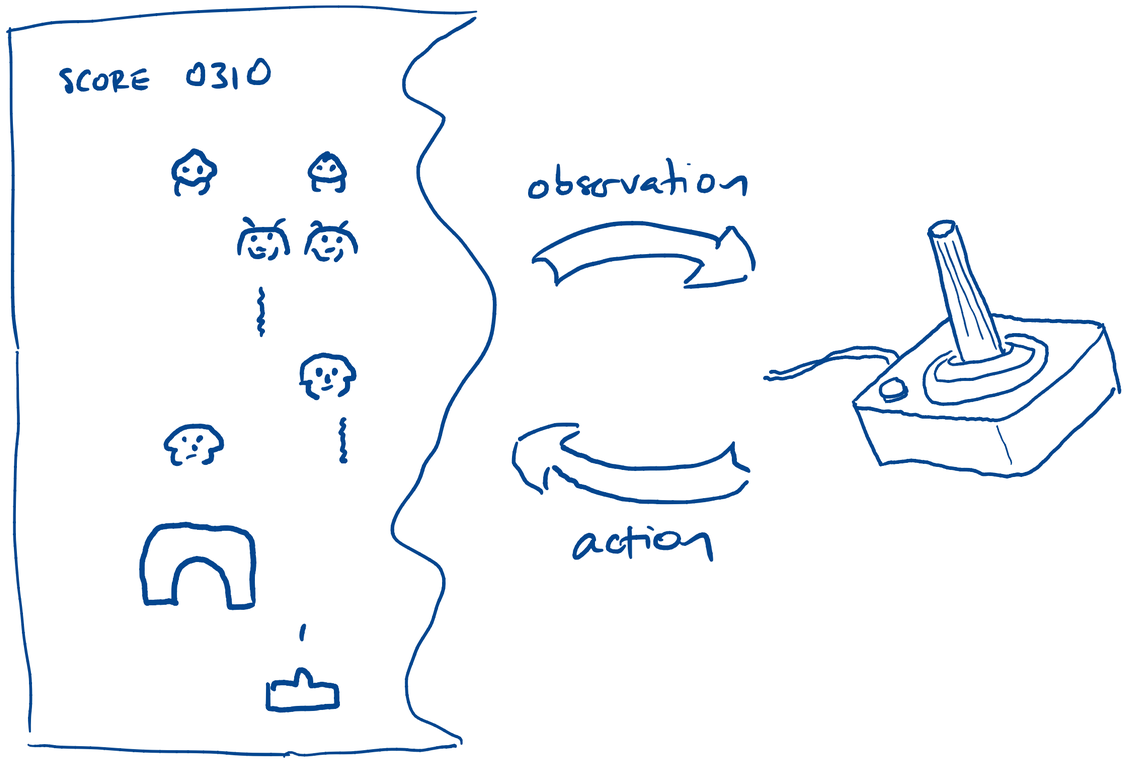
- To go beyond tabular, need some changes in our setup

No exact model



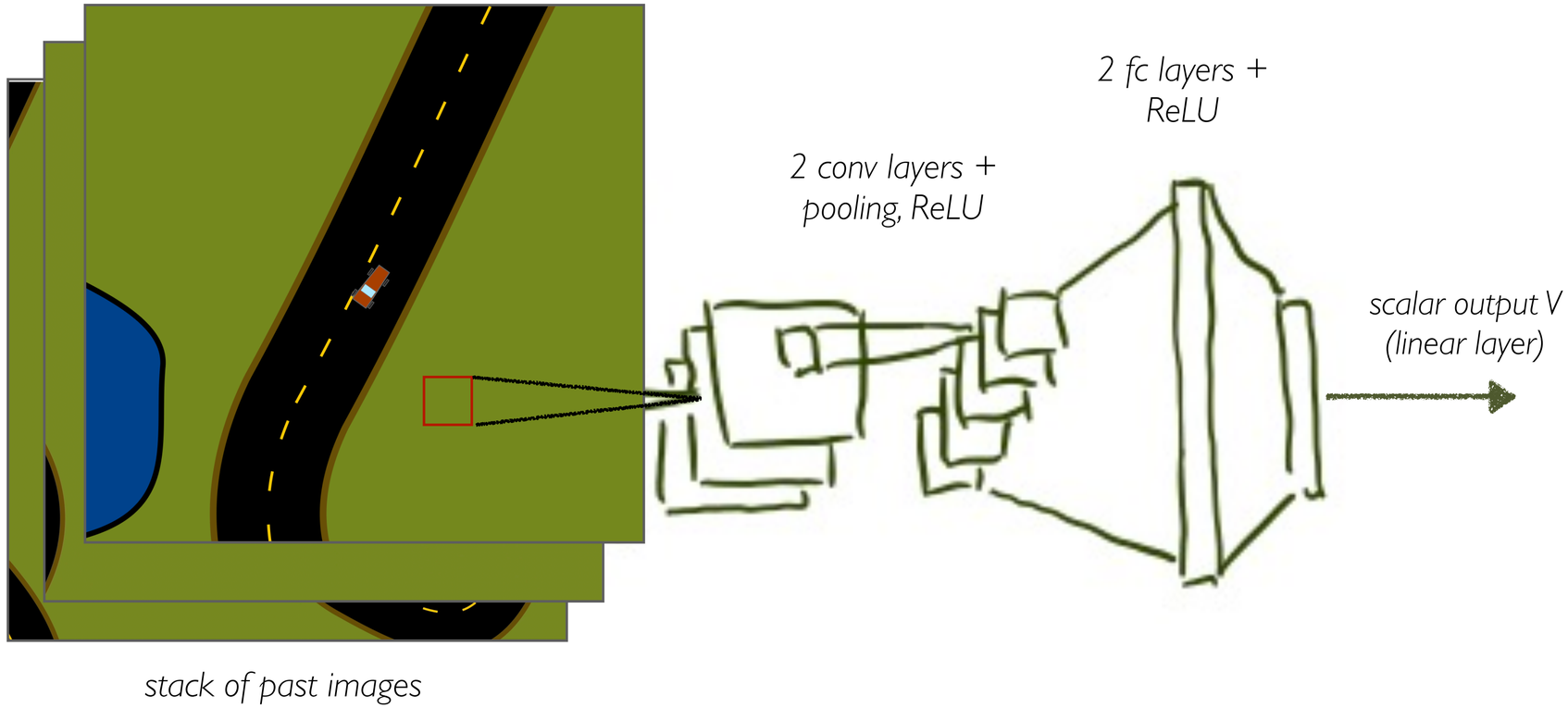
- Previously: we could write down a description of the world, e.g., expressions for $r(s, a)$ or $T(s' \mid s, a)$
- Instead: agent just interacts with environment over time — if we want $r(s, a)$ etc., have to learn it from data

No exact model



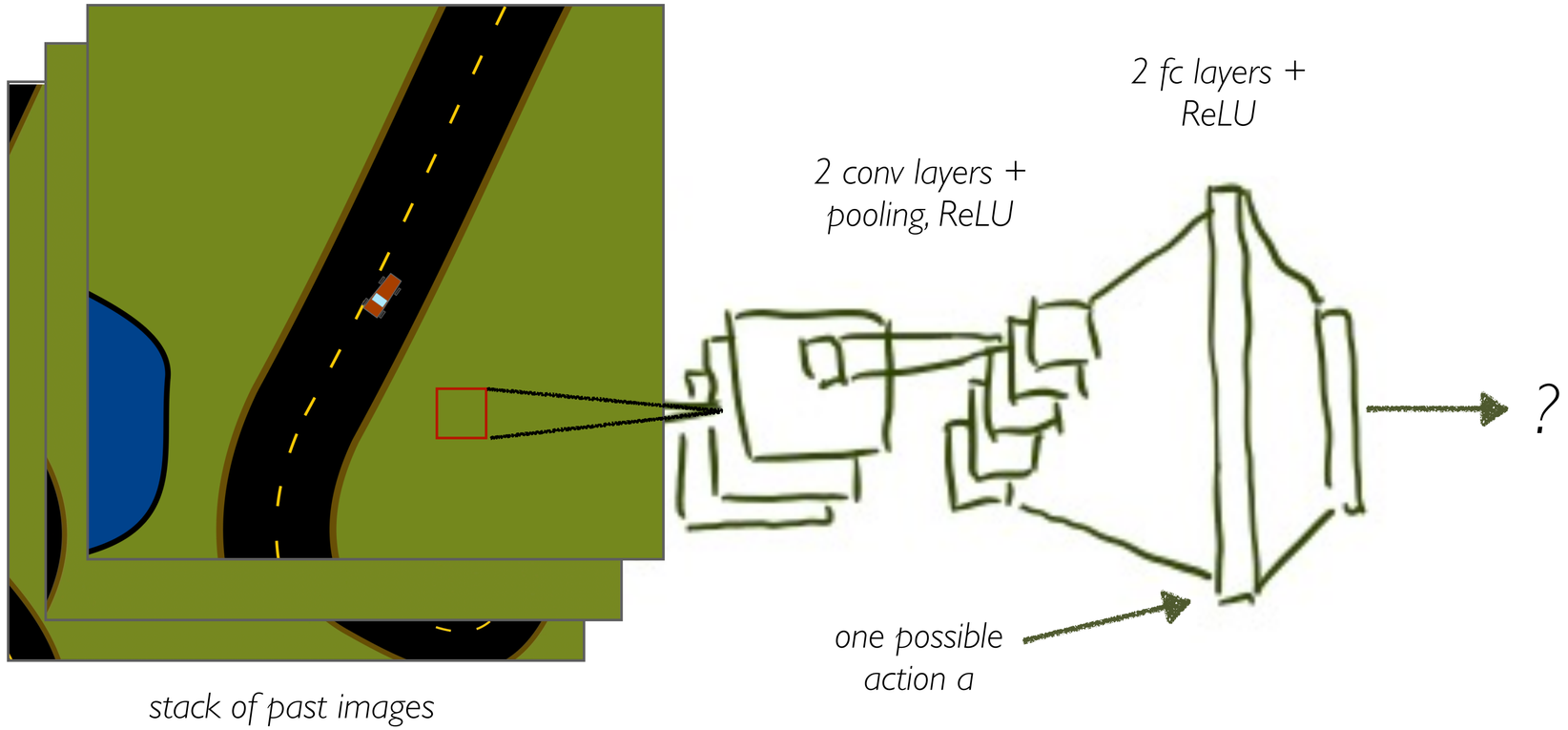
- Two broad approaches:
  - learn V, Q, etc., directly from data: *model-free RL*
  - or learn $T(s' \mid s, a)$, $r(s, a)$, etc., then pretend we had a model all along: *model-based RL*

Learned, approximate functions



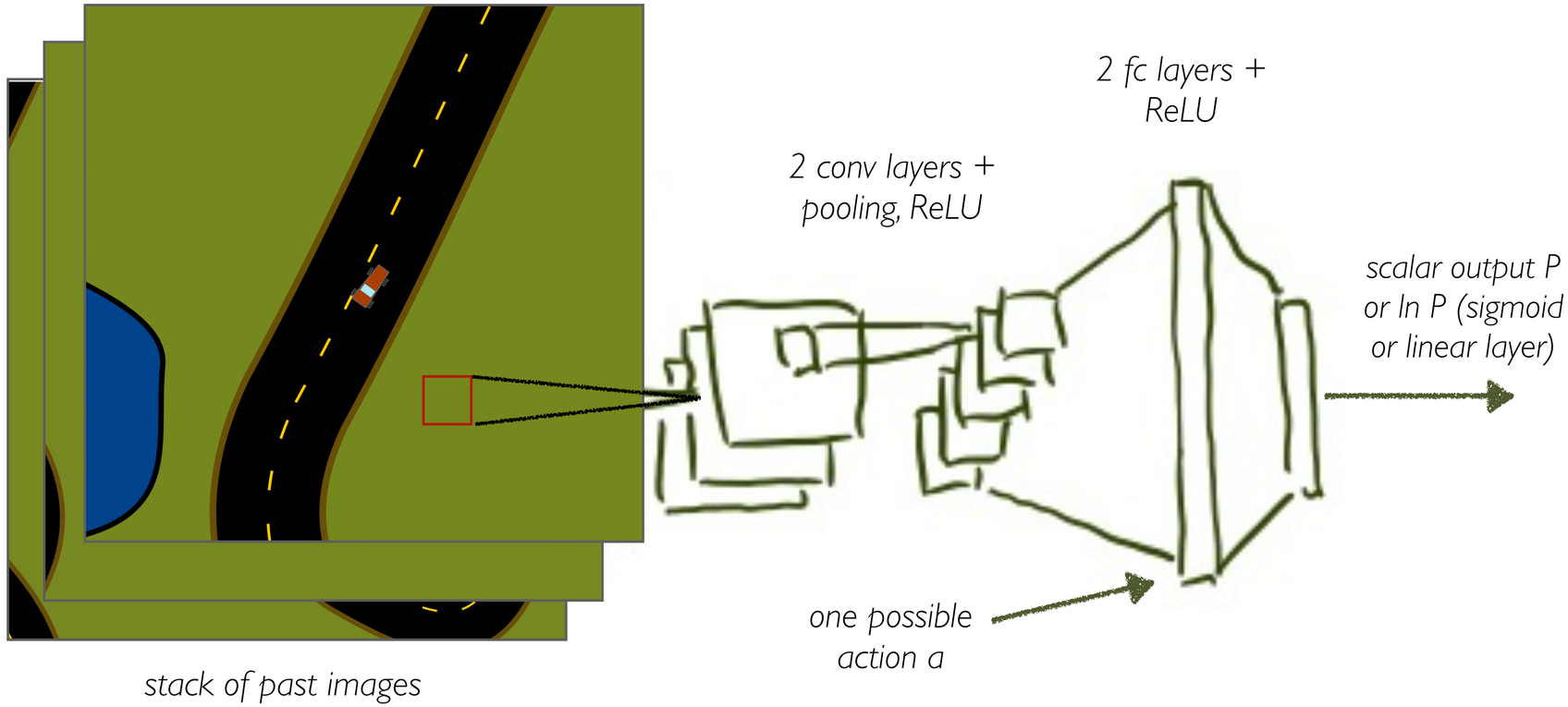
- Previously: could list out all states, keep a table of a function like $V^{\pi}(s)$
- Now: any function we care about has to be represented as an ML model, e.g., a deep net
- One parameter vector per function, each can have its own network architecture

Policy



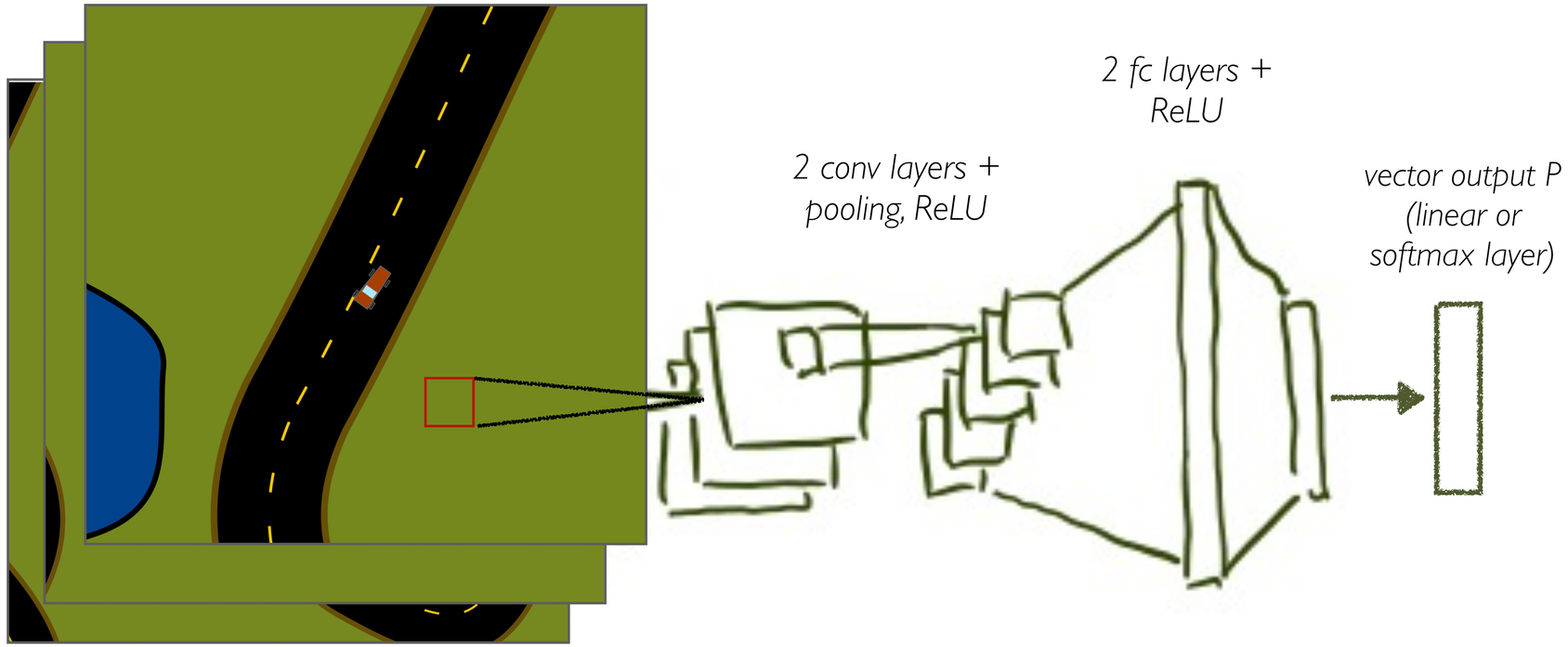
- V and Q are straightforward to represent as models: regression $\mathcal{S} \to \mathbb{R}$ or $\mathcal{S} \times \mathcal{A} \to \mathbb{R}$
- Policy is a model too: represents $P(a \mid s, \pi)$
  - recall: stochastic! (lets optimizer make small changes)
- Several common ways to set up

Policy



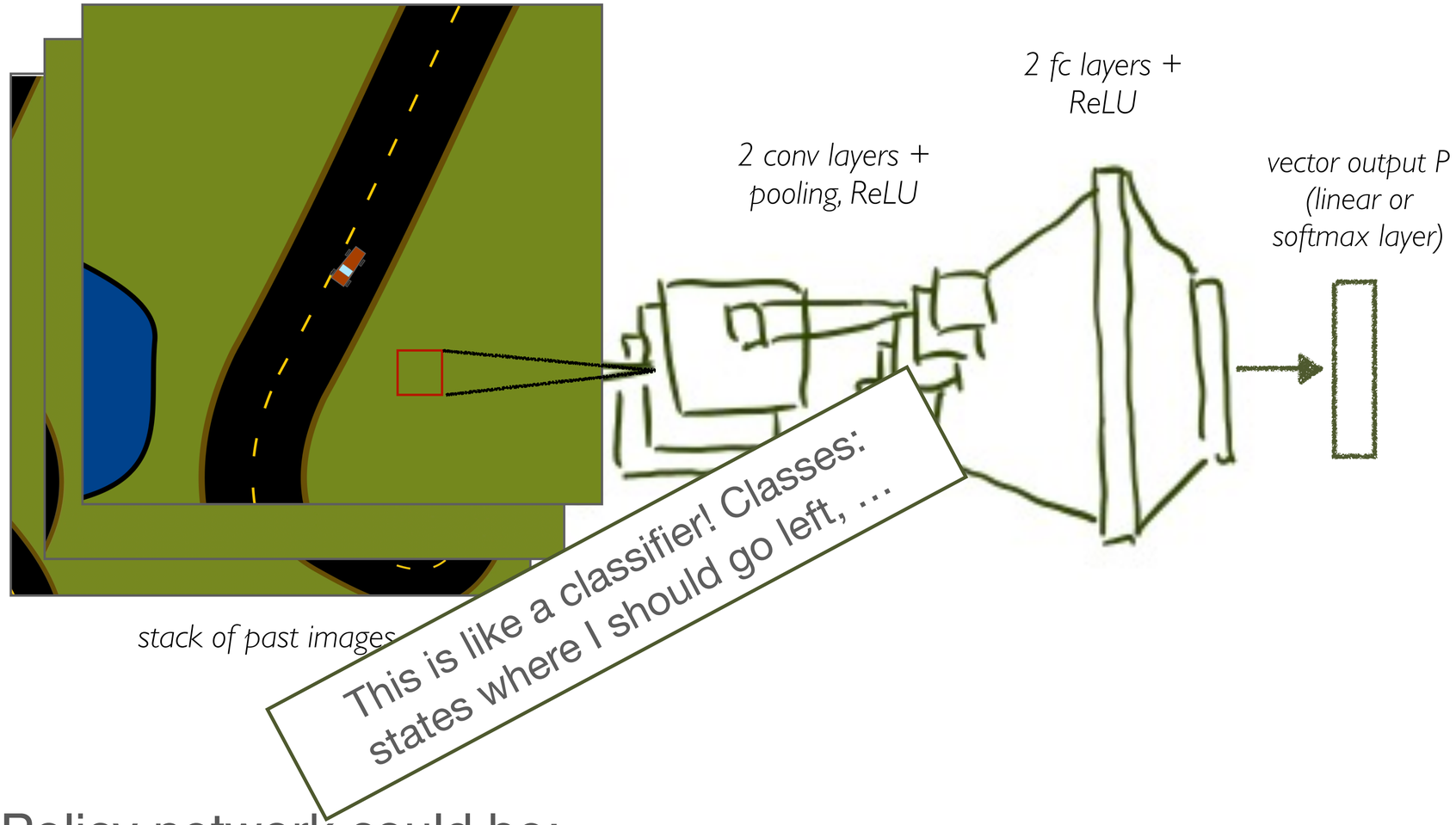
- Policy network could be:
  - $s, a \mapsto P(a \mid s, \pi)$ or $\ln P(a \mid s, \pi)$

Policy



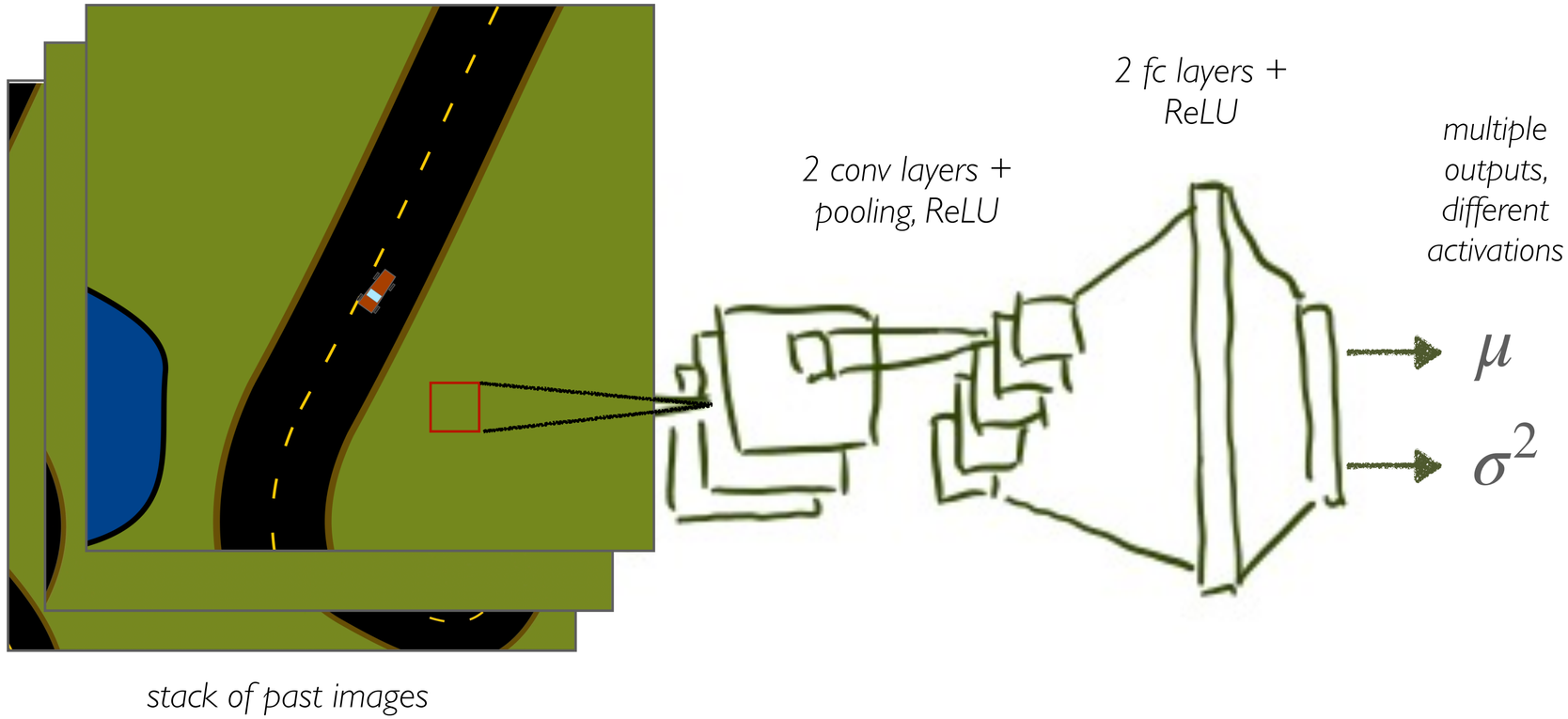
- Policy network could be:
  - $s \mapsto [P(a_1 \mid s, \pi), P(a_2 \mid s, \pi), \ldots, P(a_k \mid s, \pi)]^\top$ (or logits)

Policy



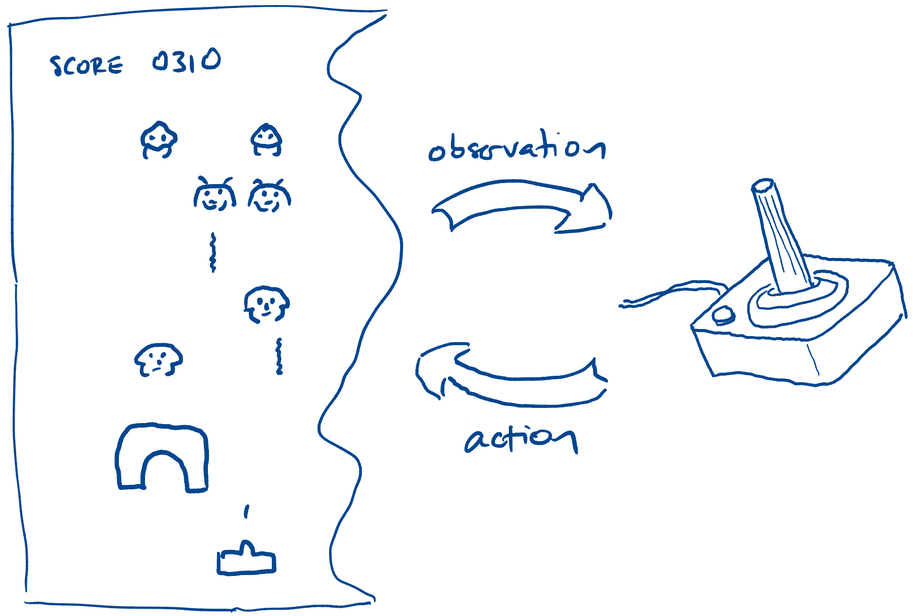
- Policy network could be:
  - $s \mapsto [P(a_1 \mid s, \pi), P(a_2 \mid s, \pi), \ldots, P(a_k \mid s, \pi)]^\top$ (or logits)

Policy



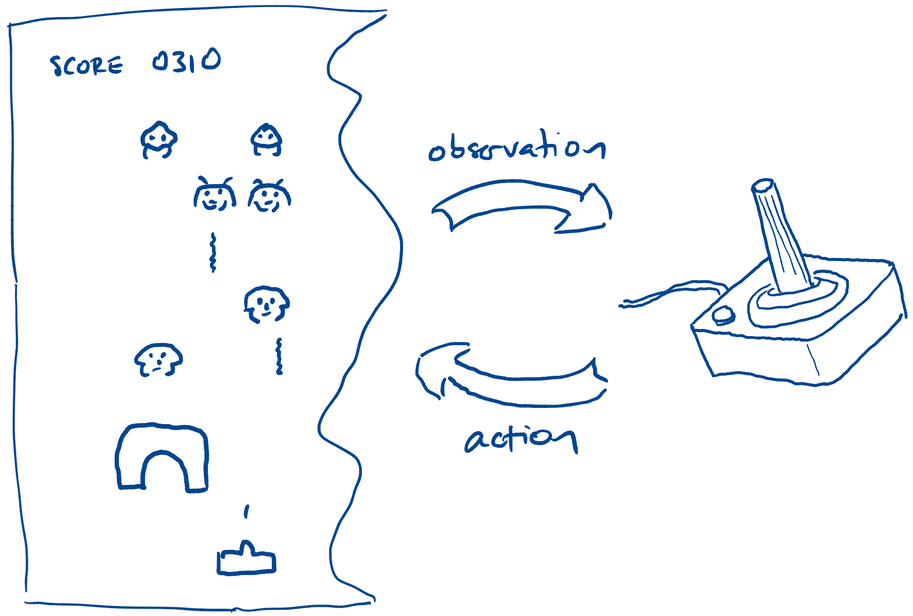
- Policy network could be:
  - $s \mapsto$ parameters of action distribution like mean, variance

State vs. observation



- As we scale up, more likely that agent doesn't see state completely and directly: $s_t \neq o_t$, partially observable
  - instead, observation informs about state: e.g., screen images help determine position and velocity
  - often need to fuse information from several o_t: e.g., velocity
- common source of bugs!

State vs. observation

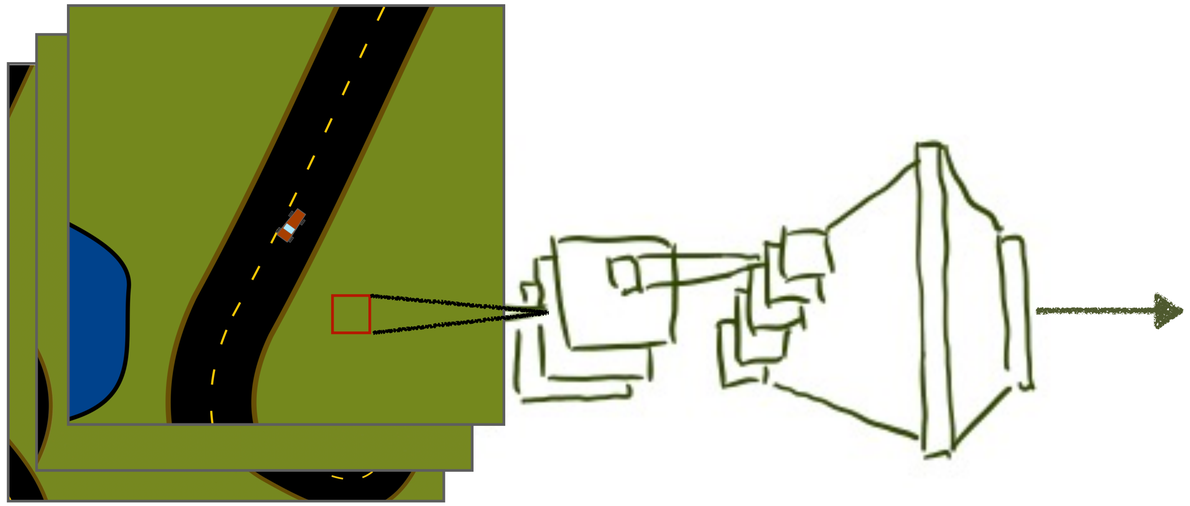


- Simplest approach: network implicitly figures out state
  - suppose network input is ***enough info to reconstruct state***
  - e.g., stack of past observations & actions (images & buttons)
  - then in order to predict V^π, some hidden activations somewhere must be a state representation
- Doesn't always work, many fancier approaches

Can't simply copy tabular algorithms

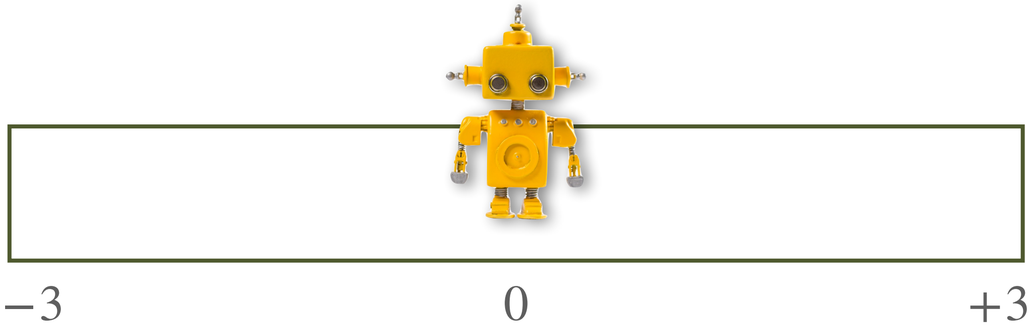
- We showed lots of dynamic programming algorithms for the tabular case — many ***do not scale***
  - fail in weird ways when we use function approximation
- In general,
  - SGD-like policy evaluation algorithms (TD for V^π, SARSA for Q^π) still work if value = output of deep net
  - algorithms w/ max (Q-learning for Q^*, value iteration for V^*) no longer work well
  - can sometimes make them to do something by force of engineering, e.g., DDQN — but not SOTA
- So what do we do instead?

Learning V^π



- Want to train a network $V^\pi_\phi(s)$ w/ parameters ϕ
  - inputs: state info, e.g., stack of images
  - output: value estimate
- Simplest approach: regression on whole-trajectory returns
- Follow π, observe trajectories $s_1, a_1, r_1, s_2, a_2, r_2, \ldots$
- Each trajectory yields several training examples

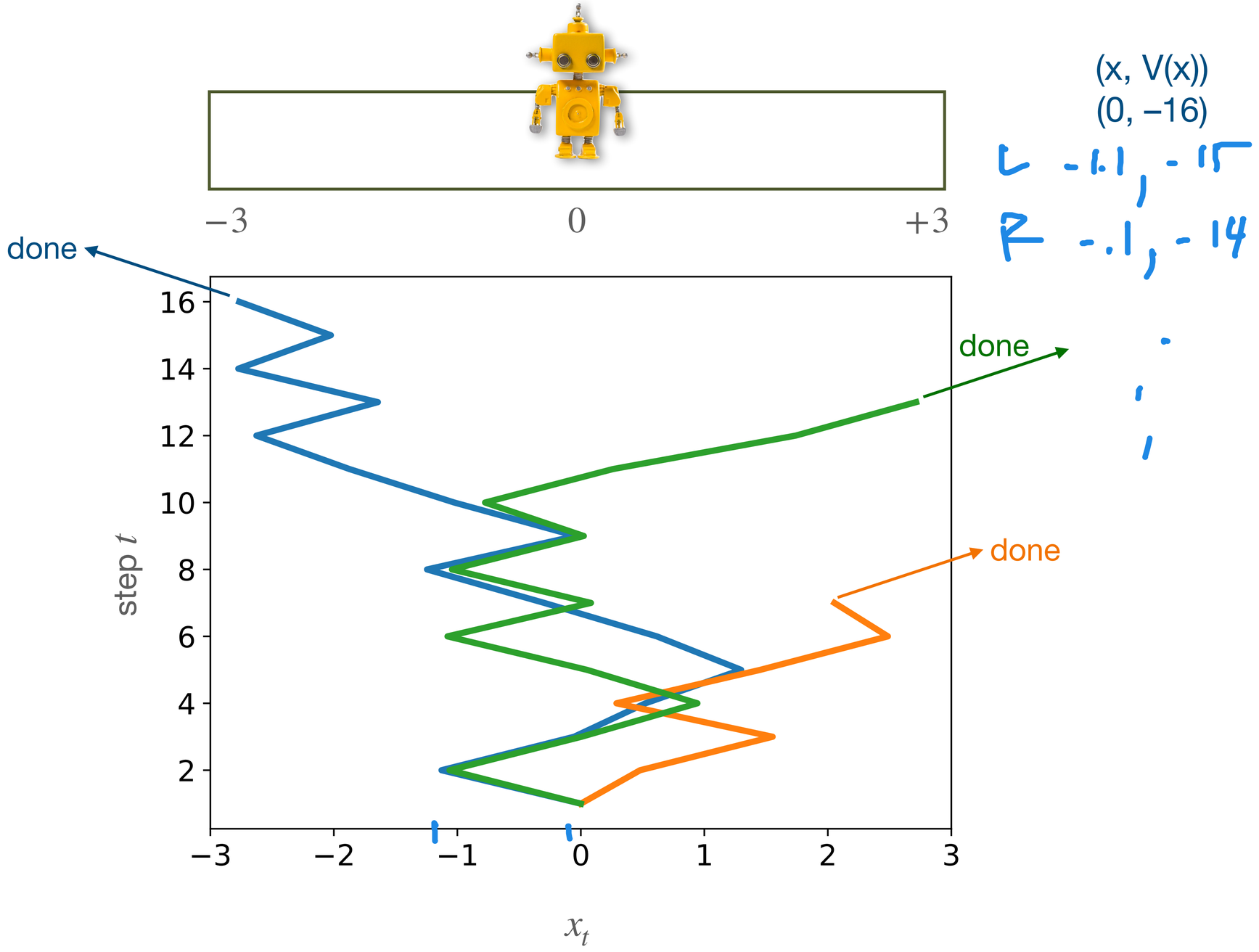
Learning V^π: example



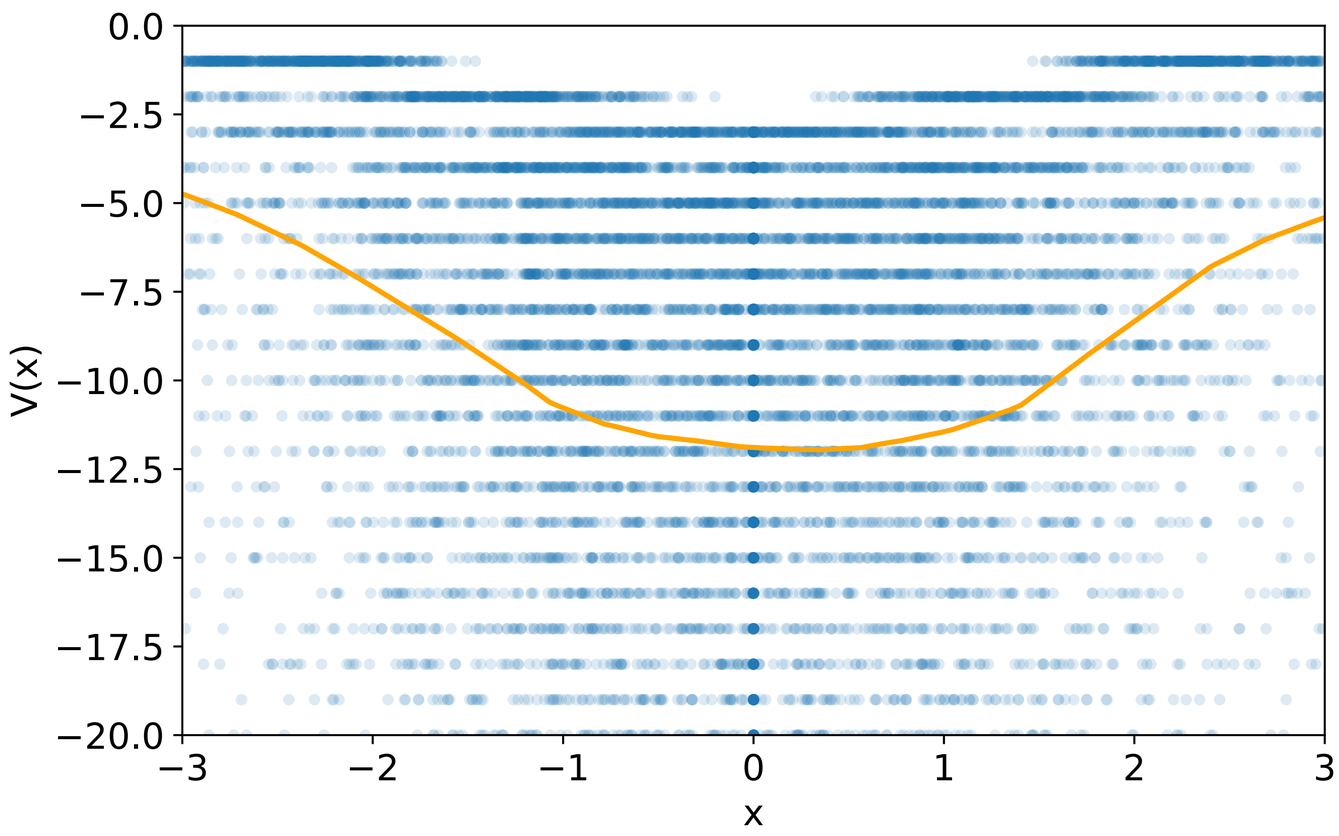
- Environment (continuous version of earlier example):
  - state $x \in [-3,3]$, start at $x = 0$
  - actions L: $x = x - N(1, \sigma^2)$ and R: $x = x + N(1, \sigma^2)$
  - rewards: –1 per action, terminate when $x \notin [-3, 3]$
  - $\gamma = 1, \sigma = \frac{1}{4}$

Some sample trajectories

Each trajectory yields several training examples



Learned V^π



- Train as supervised regression (minimize MSE)
- Blue dots: training points (1k trajectories, ~12k samples)
- Orange line: fitted V^π (2-layer ReLU net, width 64)
- Note: extremely noisy!

this is discounted version; undiscounted ($\gamma = 1$) is fine too as long as we handle termination reasonably

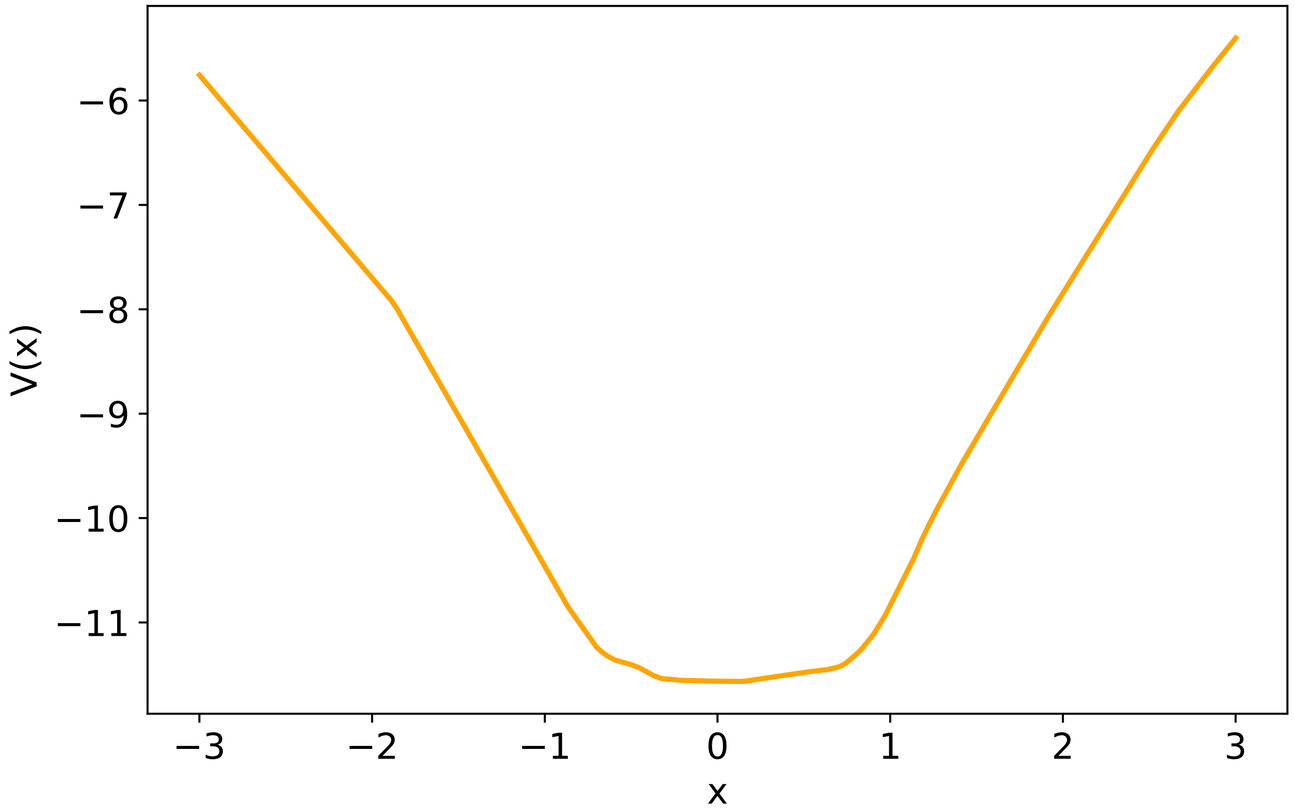
Dynamic programming

this is called TD(0), in contrast to TD(1) for previous regression method — there's a whole family TD(λ)

- 2nd try: learn V^π by dynamic programming
  - generalize tabular temporal difference learning: same algorithm but with V^π represented as a model with parameters ϕ instead of a table of numbers
- For a minibatch of sampled transitions (s, r, s')
  - with gradients off, calculate target values $r + \gamma V^\pi_\phi(s')$
  - then train each $V^\pi_\phi(s)$ towards its target
- Essentially the same code w/ model V^π_ϕ instead of table

```
with torch.no_grad():
  target = r + gamma * V(next_s)
loss = 0.5 * ((target - V(s)) ** 2).sum()
opt.zero_grad(); loss.backward(); opt.step()
```

TD(0) example




```
for _ in range(3000):
    xs, rs = trajectory()
    T = len(xs)
    with torch.no_grad():
        tgt = [rs[t] + model(xs[t+1]) for t in range(T-1)] + [rs[T-1]]
    err = 0.0
    for t in range(T):
       err += criterion(model(xs[t]), tgt[t])
    optimizer.zero_grad()
    err.backward()
    optimizer.step()
```

Learn Q^π the same ways as V^π

- Set up supervised regression problem
  - training examples map $s_t, a_t \mapsto$ sum of discounted rewards after step t (just like learning V^π but w/ a_t too)
- Or use fixed point iteration:
  - Q^π satisfies a Bellman equation just like V^π:

$$Q^\pi(s_t, a_t) \approx r_t + \gamma Q^\pi(s_{t+1}, a_{t+1})$$

  - evaluate RHS with gradients off
  - train LHS by SGD
- These are called *SARSA(1)* and *SARSA(0)*

Policy improvement

- In tabular case, can just update π to be greedy for Q^π

$$\pi^{\text{gr}}(s) = \arg\max_a Q^\pi(s, a)$$

  - could try the same here, ***but*** there will be errors in our estimate of Q^π, so switching to π^{gr} is too aggressive
- Or, in tabular case, can use max versions of backups (Q iteration, Q-learning) to directly learn Q^*
  - but doesn't work as well w/ approximate Q (DDQN kind of works but is messy and not SOTA)
  - same reason: errors in current Q estimates mean it's too aggressive to commit to greedy action
- Instead, scalable RL methods take smaller steps based on (estimates of) Q^π and V^π to improve π

How to improve our policy?

- Want to maximize $J(\theta) = \mathbb{E}_{\pi_\theta}[r_1 + \gamma r_2 + \gamma^2 r_3 + \ldots]$
- Maybe the simplest idea: analogous to SGD
  - initialize policy parameters $\theta^1 \in \mathbb{R}^d$
  - on training iteration $m = 1,2,\ldots$:
    - compute stochastic estimate $g^m \approx \frac{d}{d\theta} J(\theta)\Big|_{\theta^m}$
    - update $\theta^{m+1} \leftarrow \theta^m + \eta g^m$ (learning rate η, could be η^m)
- Called the ***policy gradient*** method
- But how do we get g^m?
  - not obvious how to differentiate expected cost J wrt θ: depends on (unknown) properties of environment

note: we're using discounted case here, but other setups are analogous

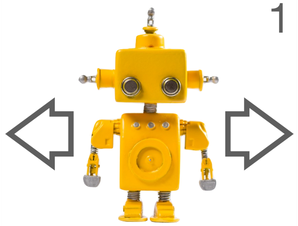
Advantage

- Def'n: ***advantage function*** is $A(s,a) = Q(s,a) - V(s)$
  - with A^π and A^* versions using Q^π, V^π and Q^*, V^*
    - A^*: how does action a compare to optimal action in s?
    - A^π: how does a compare to the distribution $\pi(a \mid s)$?
  - if $A(s,a) \geq 0$ then $Q(s,a) \geq V(s)$: a is at least as good
    - else $Q(s,a) < V(s)$: a is worse
- Intuitively, $A^\pi(s,a)$ seems related to improving π
- *Note:* still call it advantage even if we're using costs
  - "disadvantage" would be more accurate

Advantage example

$\gamma = 1$

Environment

1	2	3	4	5
				
r=–1	r=–1	r=–10	r=–1	r=0

V^*

1	2	3	4	5
–13	–12	–11	–1	0

Q^*

	1	2	3	4	5
L	–14	–14	-22	–12	0
R	–13	–12	–11	–1	0

A^*

	1	2	3	4	5
L	–1	–2	-11	–11	0
R	0	0	0	0	0

Geoff Gordon

State visitation frequency

- Define the ***visitation frequency*** for π:

$$d^\pi(s) = \mathbb{E}[\mathbb{I}(s_1 = s) + \gamma \mathbb{I}(s_2 = s) + \gamma^2 \mathbb{I}(s_3 = s) + \ldots]$$

  - where $\mathbb{I}(s_t = s)$ is 1 if we visit state s at time t
  - and zero o/w, including if episode ends before time t

- An unnormalized probability distribution
  - sum is the expected episode length for undiscounted case, or $\frac{1}{1-\gamma}$ for discounted infinite horizon

- We can write the total reward of π (what we want to optimize) as

$$J = \sum_{s,a} d^\pi(s)\, \pi(a \mid s)\, r(s,a)$$

Performance difference lemma

like expression for J but with advantage instead of reward

- The ***performance difference lemma*** formalizes the idea that $A^{\pi}(s, a)$ is related to improving our policy
- For any policies π_1 and π_2 with total rewards J_1 and J_2

$$J_1 - J_2 = \sum_{s,a} d^{\pi_1}(s)\, \pi_1(a \mid s)\, A^{\pi_2}(s, a)$$

  - to get change in total reward, visit s, a according to π_1's frequencies and evaluate A^{π_2}
- Proof sketch: $A^{\pi_2}(s, a)$ tells us how much we win each time we choose action a in s (instead of following π_2)
  - the sum is how much we win in total while following π_1